

Solved Past Paper (2023)

Short Questions

Question: 1

- 1. What is the difference between Episodic and Sequential Environment?
Explain with an example.**

Answer:

Episodic Environment:

- In an episodic environment, each decision made by an agent is independent of previous decisions.
- The agent operates in isolated episodes, where it perceives, decides, and acts without requiring memory.
- These environments are simpler because the agent does not need to plan or learn from past experiences.

Example:

In image classification, an AI system classifies each image as a cat, dog, etc., based only on the image's features—not the previous ones. Each image is a separate episode.

Sequential Environment:

- In sequential environments, actions are interdependent, and decisions made now will affect future outcomes.
- The agent must consider the history of previous states and actions. Hence, memory and planning become essential.

Example:

In robot navigation or chess, each move changes the environment and impacts future decisions. A wrong move early in a chess game can lead to failure many moves later.

Key Differences:

Aspect	Episodic	Sequential
Dependency:	No memory of past	Depends on past and future actions
Complexity:	Simpler	More complex due to planning and memory
Examples:	Email spam filtering, image tagging	Chess, driving, robotic surgery

Question: 2

2. What are included in Problem Definition? Write problem definition of any one toy problem.

Answer:

Problem Definition in AI Search consists of:

- 1. Initial State** – The starting configuration of the problem.
- 2. Action Set (Successor Function)** – All possible actions that can be applied to a state.
- 3. Transition Model** – Defines the result of performing an action in a given state.
- 4. Goal Test** – Checks whether a given state satisfies the goal condition.
- 5. Path Cost** – A function that assigns a numeric cost to a path (used for optimization).

Toy Problem Example

Vacuum Cleaner World:

1. **Initial State:** The agent is in location A or B, and either or both locations may be dirty.
2. **Actions:** Suck (clean the square), Move Left, Move Right.
3. **Transition Model:** Applying “Suck” in a dirty square makes it clean; “Move” changes position.
4. **Goal Test:** Both squares A and B are clean.
5. **Path Cost:** Each action has a cost of 1; total cost is the number of actions performed.

Why it's important:

- Clear problem definition allows AI agents to formulate the search space, apply algorithms like BFS, DFS, A*, and find optimal solutions.

Question: 3

3. Which factors determine performance of a problem-solving technique?

Answer:

The efficiency and reliability of an AI search or problem-solving technique are judged by four key performance metrics:

1. Completeness:

- Can the algorithm guarantee finding a solution if one exists?

- Example: Breadth-First Search is complete.

2. Optimality:

Does it always find the best solution with the lowest path cost?

Example: A* with an admissible heuristic is optimal.

3. Time Complexity:

- The total number of nodes expanded before finding a solution.
- Often measured as a function of depth d and branching factor b (e.g., $O(b^d)$).

4. Space Complexity:

- The maximum number of nodes stored in memory at any time.
- Critical in large or infinite state spaces.

Example Comparison:

Algorithm	Complete	Optimal	Time	Space
BFS	Yes	Yes (uniform cost)	$O(b^d)$	$O(b^d)$
DFS	No	No	$O(b^m)$	$O(bm)$
A*	Yes	Yes (with admissible $h(n)$)	Depends on heuristic	High

Question: 4

4. What is a Bidirectional Search? How is it useful?

Answer:

Bidirectional Search:

It simultaneously performs two searches:

- **Forward Search:** From the initial state toward the goal.
- **Backward Search:** From the goal state backward to the initial state.

Working Principle:

- i. Each search builds a tree of explored nodes.
- ii. When the two trees intersect, a path from start to goal is found.

Time and Space Efficiency:

- i. Traditional search time complexity is $O(b^d)$.
- ii. Bidirectional search reduces it to $O(b^{(d/2)})$, making it exponentially faster for large state spaces.

Use Case Example:

- **Route Planning in GPS:** Finding the shortest path between two cities by searching from both ends.

Challenges:

- ✓ Requires that the goal state is known and reversible actions are available (not always possible).
- ✓ Implementing backward search can be harder in complex environments.

Question: 5

5. What is game playing? How is it used as a problem-solving technique?

Answer:

Game Playing in AI:

- Refers to creating agents that can play and compete in strategic, rule-based environments.
- Games are adversarial problems – multiple agents (players) with conflicting goals.

Key Concepts:

- Minimax Algorithm:** Determines the best move by minimizing the opponent's maximum payoff.
- Game Tree:** Represents all possible moves and counter-moves.
- Heuristic Evaluation:** Estimates the utility of non-terminal game states when search cannot reach the end.

Types of Games:

- Deterministic vs. Stochastic:** With or without randomness (e.g., chess vs. backgammon).
- Perfect Information vs. Imperfect:** All players know the full state (e.g., chess vs. poker).

Example

Chess AI:

- Uses minimax with alpha-beta pruning.

- Evaluates millions of positions using heuristics.
- Learns from historical games using deep learning (like AlphaZero).

Question: 6

6. What is the difference between Weak and Strong Intelligence?

Answer:

Feature	Weak AI (Narrow AI)	Strong AI (Artificial General Intelligence)
Capability:	Task-specific, limited to one domain	General-purpose, can perform any cognitive task
Learning & Adaptation:	Learns within specific boundaries	Learns and adapts across domains
Consciousness:	No self-awareness or understanding	Hypothetical; capable of self-awareness and reasoning
Example:	Siri, Google Translate, Recommendation Systems	None exists yet; theoretical AGI models (e.g., human brain simulation)

Explanation:

- Weak AI focuses on building tools that appear intelligent (rule-based systems, ML classifiers).
- Strong AI aims to replicate human-level reasoning, problem-solving, and even consciousness.

- Currently, all deployed AI systems are weak AI.

Research Frontier:

Strong AI raises ethical and philosophical issues, such as the rights of intelligent machines or the risks of super intelligent systems

Long Questions

Question: 1

1. What is a Sensor-less Problem? Discuss with the help of an example.

Answer:

Definition:

“A sensor-less problem, also called a conformant problem, is a type of planning problem in artificial intelligence where the agent has no sensing capability and therefore does not know its current state during execution.”

It must plan actions in a way that guarantees success regardless of the true initial state or environment changes, assuming it has a complete model of how actions affect the environment.

Formal Representation:

A sensor-less problem can be defined by:

S = Set of all possible world states

A = Set of actions

T(s, a) = Transition model (what action a does in state s)

I = Set of all possible initial states

G = Goal state(s)

The agent must find an action sequence $A_1, A_2, \dots, A_n$ such that for all $s \in I$, the state after applying the sequence reaches some $g \in G$.

Example :

Vacuum World (Modified):

Imagine a vacuum cleaner in a two-room environment (Room A and Room B). The agent:

Does not know its initial location.

Cannot detect whether a room is clean or dirty (no sensors).

Its task: Ensure both rooms are clean.

Possible Initial States (Belief State):

- (At A, A dirty, B dirty)
- (At A, A clean, B dirty)
- (At B, A dirty, B dirty)
- (At B, A clean, B clean), etc.

Action Plan (Universal):

- Suck (if there's dirt)
- Move to the other room
- Suck again

Key Idea:

Although the agent doesn't know its state, this deterministic action sequence will guarantee the goal is achieved in all possible starting scenarios.

Challenges:

- ✓ Belief state tracking is required: the agent maintains a set of possible states.
- ✓ Cannot rely on feedback; must plan with certainty under uncertainty.
- ✓ Often solved using AND-OR search trees or conformant planning algorithms.

Applications:

- Robotics in partially observable environments.
- Autonomous agents in hostile or unknown terrains.
- Search-and-rescue bots without environmental sensing.

Question: 2

2. Discuss with the help of an example how A* search is better than Greedy Best-First Search.

Answer:

Introduction to Both Algorithms:

Search Type Evaluation Function

1. **A*** $f(n) = g(n) + h(n)$
2. **Greedy Best-First** $f(n) = h(n)$

Where:

$g(n)$ = cost from start node to current node n

$h(n)$ = heuristic estimate from n to goal

1. A*:

Uses both actual cost ($g(n)$) and estimated cost ($h(n)$) to make decisions.

Finds optimal solution if $h(n)$ is admissible (never overestimates).

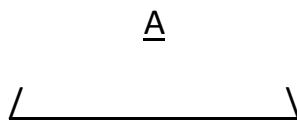
2. Greedy Best-First Search:

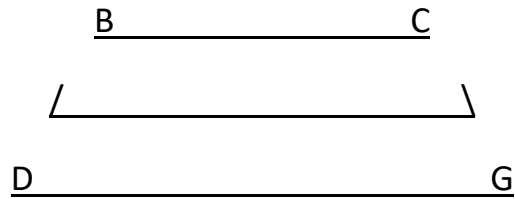
Chooses nodes that appear closest to the goal ($h(n)$ only).

Fast but not optimal, as it ignores cost already incurred.

Example:

Graph:





Node	$g(n)$	$h(n)$
A	0	4
B	1	3
C	2	2
D	2	5
G	5	0

Greedy Best-First will follow $A \rightarrow C \rightarrow G$ (based on lowest $h(n)$)

Path cost = $g(G) = 5$

A* will consider both cost so far and heuristic:

$A \rightarrow B \rightarrow D \rightarrow G$ (if that path had lower $f(n)$)

Or $A \rightarrow C \rightarrow G$ if it's optimal

Outcome:

- A* balances path cost and heuristic: ensures shortest path.
- Greedy may get trapped in local minimum or suboptimal path.

Why A* is better:

1. Optimality: A* always finds the least-cost path if $h(n)$ is admissible.
2. Completeness: A* is complete if branching factor is finite and step costs are positive.
3. Informed: A* uses more information (both cost and heuristic).

Drawback:

- A* uses more memory than greedy; may become intractable in large state spaces.
- Solutions include Iterative Deepening A* or Memory-Bounded A*.

Question: 3

3. Give brief answers of the following:

(a) Prove that A* is consistent.

Answer:

Definition:

"A heuristic $h(n)$ is consistent (monotonic) if for every node n and every successor n' generated by action a , the following holds:

$$h(n) \leq c(n, a, n') + h(n')"$$

Where:

$c(n, a, n')$ is the cost of reaching n' from n .

This implies that the estimated cost of reaching the goal from n is no more than the cost of getting to n' plus the estimated cost from n' to the goal.

Proof (A*):

A* uses $f(n) = g(n) + h(n)$

If h is consistent, then $f(n') \geq f(n)$

So, once a node is expanded, it is never re-entered with a lower cost.

This ensures no need for node revisiting, and guarantees optimality.

Hence, A* with consistent heuristic is both optimal and efficient.

(b) How can two heuristic functions be compared?

Answer:

Heuristic comparison is essential in evaluating search strategies. Two heuristics $h_1(n)$ and $h_2(n)$ can be compared based on the following:

1. Dominance:

If $h_2(n) \geq h_1(n)$ for all nodes and both are admissible, then h_2 is said to dominate h_1 .

Dominant heuristics are more informed and result in fewer node expansions.

2. Admissibility:

Both heuristics must never overestimate the true cost to be admissible.

i.e., $h(n) \leq h^*(n)$ where $h^*(n)$ is actual cost.

3. Effectiveness:

Measured by how many nodes are expanded.

Better heuristics lead to smaller search trees.

4. Consistency:

If both heuristics are consistent, they ensure no re-expansion of nodes.

Example:

For an 8-puzzle:

$h_1(n)$ = number of misplaced tiles

$h_2(n)$ = sum of Manhattan distances

Since **$h_2(n) \geq h_1(n)$** in all cases and both are admissible, **h_2** dominates **h_1** and will lead to faster search.